

Vanilla GCN

Mathematical Modelling

[Your name] [Roll no.]

Detecting illicit Bitcoin transactions on the Elliptic dataset using a 2-layer Vanilla GCN.

Part 1: The Model

Graph Representation

$$G = (V, E), \quad |V| = 203,769 \text{ (transactions)}, \quad |E| = 234,355 \text{ (money flows)}$$

Adjacency matrix: $A \in \{0, 1\}^{N \times N}$

Feature matrix: $X \in \mathbb{R}^{N \times F}, \quad F = 165$

Label: $y \in \{0, 1\}, \quad 0 = \text{licit}, 1 = \text{illicit}$

Symmetric Normalisation of the Adjacency Matrix

Self-loops: $\tilde{A} = A + I_N$

Degree matrix: $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$

Symmetric normalisation: $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$

A alone gives unequal scale to high- and low-degree nodes. Symmetric normalisation gives a fair weighted average and keeps the edge weight equal in both directions.

Feature Normalisation

$$x_{ij} \leftarrow \frac{x_{ij} - \mu_j}{\sigma_j}$$

(μ_j = mean, σ_j = standard deviation of feature j)

GCN Layer and Forward Pass

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)})$$

$H^{(l)} \in \mathbb{R}^{N \times d_l} \rightarrow$ node features entering layer l

$W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}} \rightarrow$ learnable weight matrix

$\hat{A} \rightarrow$ normalised adjacency matrix

$\sigma \rightarrow$ activation (ReLU for the hidden layer)

2-layer Vanilla GCN forward pass:

$$Z = \text{softmax}\left(\hat{A} \cdot \text{ReLU}(\hat{A} \cdot X \cdot W^{(0)}) \cdot W^{(1)}\right)$$

$$\begin{array}{llll}
N = 203,769 & F = 165 & H = 100 \text{ hidden} & C = 2 \text{ classes} \\
X & \rightarrow N \times F & XW \rightarrow \text{linear transformation} & \\
XW^{(0)} & \rightarrow N \times H & & \text{of each node's own features} \\
H^{(1)} = \text{ReLU}(\hat{A}XW^{(0)}) & \rightarrow N \times H & \hat{A}(\cdot) \rightarrow \text{aggregation} & \\
H^{(1)}W^{(1)} & \rightarrow N \times C & \sigma(\cdot) \rightarrow \text{non-linearity (ReLU)} & \\
\hat{A}(H^{(1)}W^{(1)}) = Z & \rightarrow N \times C & &
\end{array}$$

Loss Function

Weighted cross-entropy loss, computed only over labelled training nodes. Illicit transactions are rare (about 10%), so they get a higher weight: $w_1 = 0.7$ (illicit), $w_0 = 0.3$ (licit).

$$\mathcal{L} = -\frac{1}{\sum_{i \in V_{\text{train}}} w_{y_i}} \sum_{i \in V_{\text{train}}} w_{y_i} \log Z_{i,y_i}$$

Training Process

Minimise $\mathcal{L}$ with respect to the weights $\theta = \{W^{(0)}, W^{(1)}\}$ using the Adam optimiser:

$$\theta \leftarrow \theta - \eta \cdot \text{Adam update}(\nabla_{\theta} \mathcal{L}), \quad \eta = 0.01$$

$\nabla_{\theta} \mathcal{L} \rightarrow$ computed by backpropagation (chain rule).

One training epoch:

1. Forward pass on the full graph $\rightarrow Z$
2. Compute loss $\mathcal{L}$ on training nodes only
3. Backward pass $\rightarrow$ compute $\nabla_{\theta} \mathcal{L}$
4. Update weights (gradient descent step)

Repeated for 200 epochs. We train on time steps 1–34 and test on time steps 35–49 (future transactions).

Evaluation Metrics

On V_{test} (illicit = positive class):

$$\begin{aligned}
\text{Precision} &= \frac{TP}{TP + FP} & \text{Recall} &= \frac{TP}{TP + FN} \\
F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}
\end{aligned}$$

Our result: $TP = 507$, $FP = 182$, $FN = 576$

$$\text{Precision} = \frac{507}{689} = 0.736, \quad \text{Recall} = \frac{507}{1083} = 0.468, \quad F_1 = 0.572$$

Accuracy is not used because only about 6.5% of test transactions are illicit. A model that always says “licit” would be 93.5% accurate but would catch no criminal.

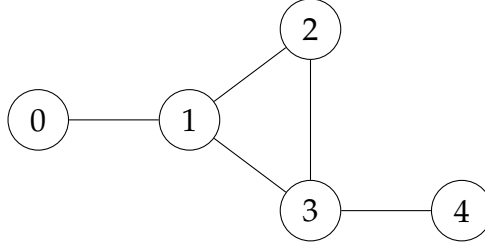
Part 2: Worked Example

1. Graph

Let $V = \{0, 1, 2, 3, 4\}$ be five transactions.

$$E = \{(0,1), (1,2), (1,3), (2,3), (3,4)\}$$

Money flows $0 \rightarrow 1 \rightarrow 2, 3 \rightarrow 4$.



Adjacency matrix, degree vector, degree matrix:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad d = [1, 3, 2, 3, 1], \quad D = \text{diag}(1, 3, 2, 3, 1)$$

2. Add Self-Loops

$$\tilde{A} = A + I = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{matrix} = 2 \\ = 4 \\ = 3 \\ = 4 \\ = 2 \end{matrix} \quad \tilde{D} = \text{diag}(2, 4, 3, 4, 2)$$

3. Symmetric Normalisation

$$\tilde{D}^{-1/2} = \text{diag}\left(\frac{1}{\sqrt{2}}, \frac{1}{2}, \frac{1}{\sqrt{3}}, \frac{1}{2}, \frac{1}{\sqrt{2}}\right)$$

Using $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$, entrywise $\hat{A}_{ij} = \frac{\tilde{A}_{ij}}{\sqrt{\tilde{d}_i \tilde{d}_j}}$:

$$\hat{A} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2\sqrt{2}} & 0 & 0 & 0 \\ \frac{1}{2\sqrt{2}} & \frac{1}{4} & \frac{1}{2\sqrt{3}} & \frac{1}{4} & 0 \\ 0 & \frac{1}{2\sqrt{3}} & \frac{1}{3} & \frac{1}{2\sqrt{3}} & 0 \\ 0 & \frac{1}{4} & \frac{1}{2\sqrt{3}} & \frac{1}{4} & \frac{1}{2\sqrt{2}} \\ 0 & 0 & 0 & \frac{1}{2\sqrt{2}} & \frac{1}{2} \end{bmatrix} \approx \begin{bmatrix} 0.5 & 0.354 & 0 & 0 & 0 \\ 0.354 & 0.25 & 0.289 & 0.25 & 0 \\ 0 & 0.289 & 0.333 & 0.289 & 0 \\ 0 & 0.25 & 0.289 & 0.25 & 0.354 \\ 0 & 0 & 0 & 0.354 & 0.5 \end{bmatrix}$$

($\hat{A}$ is symmetric, as required, since $\tilde{A}$ and $\tilde{D}$ are symmetric.)

4. Feature Matrix

Two features per transaction, $X \in \mathbb{R}^{5 \times 2}$: f_1 = unusual output pattern, f_2 = normal fee pattern.

Node	f_1	f_2
0	1	0
1	1	1
2	1	0
3	0	1
4	0	1

$$X = H^{(0)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{5 \times 2}$$

5. Labels

$C = 2$. Nodes 0 and 4 are labelled training nodes:

$y_0 = \text{class 1 (illicit)}$, $y_4 = \text{class 0 (licit)}$

One-hot: $y_0 = [0, 1]$, $y_4 = [1, 0]$. The remaining nodes are unlabelled during training.
 $V_L = \{0, 4\}$

6. Vanilla GCN Propagation Rule

$$H^{(k+1)} = \sigma(\hat{A}H^{(k)}W^{(k)})$$

With $H^{(0)} = X$ and $W^{(0)} = I_2$ (identity: the first layer only mixes neighbourhoods, with no feature transform):

$$H^{(1)} = \text{ReLU}(\hat{A}XI) = \text{ReLU}(\hat{A}X)$$

Computation of $H^{(1)} = \hat{A}X$, $(\hat{A}X)_{ij} = \sum_k \hat{A}_{ik}X_{kj}$:

$$\begin{aligned} H_0^{(1)} &= \frac{1}{2}(1, 0) + \frac{1}{2\sqrt{2}}(1, 1) = \left(\frac{1}{2} + \frac{1}{2\sqrt{2}}, \frac{1}{2\sqrt{2}}\right) \\ H_1^{(1)} &= \frac{1}{2\sqrt{2}}(1, 0) + \frac{1}{4}(1, 1) + \frac{1}{2\sqrt{3}}(1, 0) + \frac{1}{4}(0, 1) = \left(\frac{1}{2\sqrt{2}} + \frac{1}{4} + \frac{1}{2\sqrt{3}}, \frac{1}{2}\right) \\ H_2^{(1)} &= \frac{1}{2\sqrt{3}}(1, 1) + \frac{1}{3}(1, 0) + \frac{1}{2\sqrt{3}}(0, 1) = \left(\frac{1}{2\sqrt{3}} + \frac{1}{3}, \frac{1}{\sqrt{3}}\right) \\ H_3^{(1)} &= \frac{1}{4}(1, 1) + \frac{1}{2\sqrt{3}}(1, 0) + \frac{1}{4}(0, 1) + \frac{1}{2\sqrt{2}}(0, 1) = \left(\frac{1}{4} + \frac{1}{2\sqrt{3}}, \frac{1}{2} + \frac{1}{2\sqrt{2}}\right) \\ H_4^{(1)} &= \frac{1}{2\sqrt{2}}(0, 1) + \frac{1}{2}(0, 1) = \left(0, \frac{1}{2} + \frac{1}{2\sqrt{2}}\right) \end{aligned}$$

All values are ≥ 0 , so ReLU makes no changes.

$$H^{(1)} \approx \begin{bmatrix} 0.854 & 0.354 \\ 0.892 & 0.500 \\ 0.622 & 0.577 \\ 0.539 & 0.854 \\ 0 & 0.854 \end{bmatrix}$$

7. Output (Classification Layer)

Take a second weight matrix so that the two scores are exact negatives of one another:

$$W^{(1)} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}, \quad Z = \hat{A}H^{(1)}W^{(1)}$$

Compute in two stages: $M = \hat{A}H^{(1)}$, then $Z = MW^{(1)}$.

$$M_0 = \frac{1}{2}(0.854, 0.354) + \frac{1}{2\sqrt{2}}(0.892, 0.5) = (0.742, 0.354)$$

$$M_4 = \frac{1}{2\sqrt{2}}(0.539, 0.854) + \frac{1}{2}(0, 0.854) = (0.190, 0.729)$$

Doing the same for all rows:

$$M \approx \begin{bmatrix} 0.742 & 0.354 \\ 0.839 & 0.630 \\ 0.620 & 0.583 \\ 0.537 & 0.807 \\ 0.190 & 0.729 \end{bmatrix}$$

$Z = MW^{(1)}$: each row satisfies $Z_{i0} = M_{i1} - M_{i0} = -Z_{i1}$.

$$Z \approx \begin{bmatrix} -0.389 & 0.389 \\ -0.209 & 0.209 \\ -0.037 & 0.037 \\ 0.270 & -0.270 \\ 0.538 & -0.538 \end{bmatrix}$$

8. Softmax Reduces to a Sigmoid

Because the two scores in every row are exact negatives, $Z_{i0} = -Z_{i1} = -z$, the two-class softmax is

$$P_{i1} = \frac{e^z}{e^{-z} + e^z} = \frac{1}{1 + e^{-2z}}$$

which is a plain sigmoid of $2z$.

Labelled node 0 ($z = Z_{0,1} = 0.389$):

$$P_{0,1} = \frac{1}{1 + e^{-0.777}} = \frac{1}{1 + 0.460} \approx 0.685, \quad P_{0,0} \approx 0.315$$

$$P_0 \approx [0.315, 0.685] \rightarrow \text{predicted illicit} \checkmark$$

Labelled node 4 ($z = Z_{4,1} = -0.538$):

$$P_{4,1} = \frac{1}{1 + e^{1.076}} = \frac{1}{1 + 2.934} \approx 0.254, \quad P_{4,0} \approx 0.746$$

$$P_4 \approx [0.746, 0.254] \rightarrow \text{predicted licit} \checkmark$$

Unlabelled node 1 gets $P_1 \approx [0.397, 0.603]$, so it is flagged as illicit because it receives money from the illicit node 0.

9. Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{|V_L|} \sum_{i \in V_L} \sum_c y_{ic} \log P_{ic} = \frac{1}{2} [-\log P_{0,1} - \log P_{4,0}]$$

$$\begin{aligned}
y_0 = [0, 1] : \quad \ell_0 &= -\log(0.685) \approx 0.378 \\
y_4 = [1, 0] : \quad \ell_4 &= -\log(0.746) \approx 0.293 \\
\mathcal{L} &= \frac{0.378 + 0.293}{2} \approx \underline{\underline{0.336}}
\end{aligned}$$

10. Gradient of the Softmax Cross-Entropy

$$\begin{aligned}
P_{ic} &= \frac{e^{Z_{ic}}}{\sum_{c'} e^{Z_{ic'}}} & \ell_i &= -\sum_c y_{ic} \log P_{ic} \\
\frac{\partial P_{ic}}{\partial Z_{ic}} &= P_{ic}(1 - P_{ic}), & \frac{\partial P_{ic}}{\partial Z_{id}} &= -P_{ic}P_{id} \quad (c \neq d)
\end{aligned}$$

Combining:

$$\frac{\partial \ell_i}{\partial Z_{ic}} = P_{ic} - y_{ic}$$

$$\text{For node 0: } \frac{\partial \ell_0}{\partial Z_{0,0}} = 0.315 - 0 = 0.315, \quad \frac{\partial \ell_0}{\partial Z_{0,1}} = 0.685 - 1 = -0.315$$

$$\text{For node 4: } \frac{\partial \ell_4}{\partial Z_{4,0}} = 0.746 - 1 = -0.254, \quad \frac{\partial \ell_4}{\partial Z_{4,1}} = 0.254 - 0 = 0.254$$

11. Backpropagation to $\partial \mathcal{L} / \partial W^{(1)}$

With $\mathcal{L} = \frac{1}{2}(\ell_0 + \ell_4)$, only labelled rows carry a gradient $\frac{1}{2}(P_i - y_i)$; unlabelled rows are 0.

$$dZ = \begin{bmatrix} 0.157 & -0.157 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ -0.127 & 0.127 \end{bmatrix}$$

Gradient with respect to $W^{(1)}$: since $Z = MW^{(1)}$, $\frac{\partial \mathcal{L}}{\partial W^{(1)}} = M^T dZ = M_0^T dZ_0 + M_4^T dZ_4$

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} \approx \begin{bmatrix} 0.093 & -0.093 \\ -0.037 & 0.037 \end{bmatrix}$$

A gradient descent step

$$W^{(k)} \leftarrow W^{(k)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(k)}}$$

completes one full training iteration of the Vanilla GCN.